

OmniRAG: A Multi-Agent Knowledge Intelligence Platform for Retrieval, Reasoning, and Automation

Literature Review & Work Distribution Report

Literature Survey Strategy

The proposed project integrates several state-of-the-art technologies, including Retrieval-Augmented Generation (RAG), Multi-Agent Systems, hierarchical document retrieval, workflow orchestration, and intelligent knowledge management. To ensure comprehensive understanding and efficient development, the literature survey has been divided among three team members according to their respective project responsibilities.

Rather than reviewing unrelated research papers, each member studies literature directly aligned with the module they will design and implement. This approach allows every component of OmniRAG to be developed using well-established research while contributing to a unified system architecture.

Team Member 1: Core Systems & Retrieval

NAME: Pranjal Agarwal

ASSIGNED MODULE: Document Intelligence and Retrieval Engine

Module Overview & Key Responsibilities:

Pranjal Agarwal is responsible for designing and implementing the document processing and intelligent retrieval subsystem, which serves as the foundational data layer of the OmniRAG platform.

- Document upload, structural parsing, and metadata management
- Heading/structure detection and intelligent document segmentation
- Chunk optimization and embedding generation pipelines
- Hierarchical indexing structures and semantic search mechanics
- Dynamic temporary high-resolution vector store generation for long documents

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks Authors: Patrick Lewis et al. Year: 2020
REVIEW SUMMARY Lewis et al. introduced the Retrieval-Augmented Generation (RAG) framework, which combines dense document retrieval with sequence generation to improve the factual accuracy of Large Language Models. Instead of depending solely on the internal knowledge of a language model, the proposed framework retrieves relevant information from an external knowledge source before generating a response. The research demonstrated significant improvements in open-domain question answering by integrating retrieval with generation, substantially reducing hallucinations.
LIMITATIONS IDENTIFIED Single-stage retrieval process; flat document indexing structure; limited optimization for exceptionally long documents; lack of task planning or collaborative multi-agent reasoning.
CONTRIBUTION TO OMNIRAG Provides the absolute theoretical foundation of the OmniRAG retrieval pipeline. The concepts of semantic retrieval, vector embeddings, and context-aware response generation are directly adopted and extended within the proposed architecture.

Literature Review Paper 2

RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval
Year: 2024

REVIEW SUMMARY

RAPTOR introduces a hierarchical retrieval mechanism designed specifically for long and complex documents. Instead of storing all document chunks equally, RAPTOR recursively summarizes sections and organizes them into a tree-like hierarchy, allowing retrieval to begin from higher-level summaries before navigating down to granular, detailed content. This significantly improves retrieval efficiency and context mapping.

LIMITATIONS IDENTIFIED

Primarily focuses on document retrieval logic; does not incorporate autonomous agent collaboration; lacks integrated custom computation or workflow orchestration mechanisms.

CONTRIBUTION TO OMNIRAG

Directly inspires OmniRAG's hierarchical document indexing strategy. Instead of embedding entire documents into a single flat vector database, OmniRAG utilizes heading-level indexing to identify relevant document sections first, creating high-resolution vector stores dynamically for selected segments to optimize latency and computational overhead.

Team Member 2: Agent Orchestration & Logic

NAME: Aryan Patel

ASSIGNED MODULE: Multi-Agent Intelligence and Reasoning Engine

Module Overview & Key Responsibilities:

Aryan Patel will design and implement the intelligent agent ecosystem responsible for distributed reasoning, query processing, and collaborative task execution.

- Designing specialized agents (Planner, Information Retrieval, Evaluator, Summarization)
- Inter-agent communication protocols and query decomposition workflows
- Dynamic tool invocation framework and external API integration
- Short-term and long-term agent memory management architectures
- Ensuring explainable reasoning trails throughout the execution lifecycle

Literature Review Paper 1

AutoGen: Enabling Next-Generation LLM Applications via Multi-Agent Conversation

Authors: Microsoft Research | Year: 2023

REVIEW SUMMARY

AutoGen presents a robust framework for constructing AI applications through collaboration among multiple specialized agents. Each agent performs a dedicated task while communicating with others to solve complex workflows. The framework natively supports autonomous planning, tool usage, code execution, and structured human-in-the-loop interaction.

LIMITATIONS IDENTIFIED

Functions as a general-purpose framework; provides limited focus on targeted document retrieval optimization; lacks domain-specific knowledge state management out-of-the-box.

CONTRIBUTION TO OMNIRAG

Forms the operational foundation of OmniRAG's Multi-Agent Intelligence Engine. The segregation of duties among the Planner, Retrieval, Evaluator, and Summarizer agents directly extends AutoGen's multi-agent conversational design patterns.

Literature Review Paper 2

ReAct: Synergizing Reasoning and Acting in Language Models

Authors: Shunyu Yao et al. | Year: 2022

REVIEW SUMMARY

ReAct introduces an elegant execution framework that alternates intermediate reasoning steps ("thinking") with external system actions ("acting"), such as database queries, web searches, or tool executions. This pattern drastically enhances real-world decision-making transparency and problem-solving tracking for multi-step tasks.

LIMITATIONS IDENTIFIED

Primarily a prompt-driven single-agent architecture; lacks a native multi-agent macro workflow orchestration system; provides minimal native document management capabilities.

CONTRIBUTION TO OMNIRAG

Directly motivates OmniRAG's Planner Agent logic, enabling complex user queries to be decomposed systematically into actionable sub-tasks. It also guides the interaction loop allowing agents to seamlessly call code runtimes and semantic index tools.

Team Member 3: Full-Stack Platform & State

NAME: Aditya Giri

ASSIGNED MODULE: Platform Development, Workflow Orchestration and Knowledge Services

Module Overview & Key Responsibilities:

Aditya Giri is tasked with full platform orchestration, state tracking, and building the user-facing secure application layer that integrates the underlying AI modules.

- User authentication, role-based access control, and session management
- Interactive dashboard interfaces for document ingestion and knowledge analytics
- Real-time visualization engines for agent reasoning graphs and workflows
- Isolated secure Python execution environments for analytical computation
- Automated report generation utilities and unified API service presentation

Literature Review Paper 1

Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection

Year: 2024

REVIEW SUMMARY

Self-RAG optimizes traditional retrieval-augmented generation architectures by dynamically training language models to critique their own generation quality and decide autonomously when external retrieval is strictly necessary, maximizing efficiency and factual alignment.

LIMITATIONS IDENTIFIED

Demands high token and computational overhead; centers tightly around generation quality critiques rather than full-scale stateful workspace orchestration.

CONTRIBUTION TO OMNIRAG

Inspires the self-reflection and assessment loops built inside OmniRAG's Evaluator Agent, establishing a validation step that critiques retrieved evidence quality before rendering answers.

Literature Review Paper 2

LangGraph: Stateful Multi-Agent Workflow Orchestration

Developer: LangChain Ecosystem

REVIEW SUMMARY

LangGraph provides a stateful, graph-based framework explicitly designed to model circular, looping, and complex conditional workflows between multiple independent LLM agents. It natively handles execution states, shared conversational memory banks, and complex conditional edge routing.

LIMITATIONS IDENTIFIED

Focuses purely on developer orchestration logic; lacks out-of-the-box knowledge services, parsing pipelines, or end-user UI presentation views.

CONTRIBUTION TO OMNIRAG

Acts as the architectural backbone for OmniRAG's runtime execution. It orchestrates multi-agent conversation state, captures intermediate agent reasoning outputs, and ensures smooth cyclical execution transitions across components.

Integrated Research Contribution

The six selected research papers collectively establish the complete theoretical and technical blueprint of OmniRAG:

- **RAG & RAPTOR:** Formulate advanced foundations for deep semantic retrieval and recursive tree-structured document indexing.
- **AutoGen & ReAct:** Provide the structural framework for multi-agent delegation, reasoning trails, and systemic tool execution.
- **Self-RAG & LangGraph:** Introduce essential self-reflection logic and stateful cyclical graph-routing orchestration.

By unifying these paradigms, OmniRAG evolves beyond generic chat-with-pdf applications into a comprehensive enterprise-ready Multi-Agent Knowledge Intelligence Platform. The alignment of specific literature to core developmental responsibilities guarantees that each module leverages academic best practices while driving forward an integrated ecosystem.